06_ DEEP RESIDUAL LEARNING

11 March 2026 16:10

They say training deep neural nets is difficult and hence introduce a residual learning framework to ease the training process

Explicitly reformulate the layers as learning residual functions w.r.t to layer inputs, instead of learning unreferenced functions

these residual networks are easier to optimize and can gain accuracy from considerably increased depth

The depth of representations is of central importance for many visual recognition tasks

They claim

Performance ∝ depth

But only if optimization works

Introduction

Deeper the network ∝ results

Deeper networks have higher training errors

exploding & vanishing gradient problems have already been addressed using normalization & better initialization so that deeper networks can start converging

training error reflects optimization, not generalization

if a training error increases its not overfitting

just that deeper models fail to optimize here

suppose a 20 layer network performs well

then a 56 layer should do better considering the 20 layer one being a subset

the extra layers should learn better identity mappings

so performance should ideally not degrade

But 56 layer really performs worse!

degradation problem

A deeper network doesn't have to perform better than a shallower one, it shouldn't just perform worse than it

More parameters increase capacity

But depth increases compositional structure

look at breadth/width/parameters

more neurons → more basis functions

Depth → more layers

→ more compositions of transformations

$x_{l+1} = x_l + G(x)$ → identity mapping

progressive refinement

The original signal always becomes accessible

They aren't saying Deep networks are theoretically bad

Instead they say optimization becomes difficult as depth increases

The issue is training dynamics and not representational capacity

Deeper networks should represent complex functions

Yet they empirically train worse!

Architectural Reformulation

Instead of stack layers → learn $H(x)$

they propose

stack layers → learn $F(x)$

Output = $F(x) + x$

Standard networks learn $H(x)$

→ input x → layers → output $H(x)$

Network directly learns mapping

instead of $H(x)$ they learn $F(x) = H(x) - x$

So output becomes $H(x) = F(x) + x$

The architecture explicitly computes

input x

$F(x)$ stack of layers

$\oplus$

output

it is not referencing inputs

it is forcing the network to learn deviations from identity

Conceptual intuition → suppose the optimal mapping is close to identity

$H(x) \approx x$ for a normal net to learn $H(x) = x$ can be hard

Residual formulation instead learns $F(x) = 0$

(much easier for optimization)

when output is $y = F(x) + x$

what small correction should I apply to the input?

gradient becomes $\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + 1$ → This identity path will ensure the gradient flows

Why residual learning helps training?

not because it references input

↳ it makes identity mappings easy to represent

stabilizes gradient flow

It converts the problem into learning residual corrections

Real optimization Problem

suppose the network needs to learn $H(x) = x$

Because the identity signal is preserved, the network can safely learn small modifications instead of entire new mappings

Standard network → layers must learn full transformations

Residual style architecture → layers only learn corrections

↳ Since these are smaller functions optimization becomes easier

Identity mapping

A standard deep network must learn

$f_L(f_{L-1}(\dots f_1(x))) = x$

But each layer does

$x_{l+1} = \sigma(W_l x_l)$

So to represent identity every layer must behave like

$\sigma(W_l x) \approx x$ → W_ls must be tuned carefully

non linearities must not distort the signal

errors compound across depth

Gradient Perspective

When training a deep network

$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial x_{L+1}} \prod_{l=1}^{L} \frac{\partial x_{l+1}}{\partial x_l}$$

Gradients multiply through many jacobians

If those jacobians have singular values < 1

↳ gradients shrink exponentially → vanishing gradients

if > 1 gradients explode

revamp → desired mapping as $H(x)$ → true function that network should learn

Standard network → layers learn $H(x)$ directly

Instead they define $F(x) = H(x) - x$

So the network learns: $F(x)$

and the final output becomes $H(x) = F(x) + x$

again restating if $H(x) = x$ is an optimal mapping

then $F(x) = 0$ ie residual function becomes zero

So the stack of non linear layers effectively does nothing

↳ this would be easier than forcing many non linear layers to produce identity

They say if optimal function is closer to identity function than zero

it means $H(x) = x + \epsilon(x)$

small

then $F(x) = \epsilon(x)$

network only needs to learn small corrections instead of learning whole mapping from scratch

Residual functions have small responses

↳ $F(x)$ usually has small magnitude

↳ meaning layers mostly learn small refinements, not large transformations

* They explicitly say shortcut connections perform identity mappings

Asymptotically Approximate complicated functions

given enough layer parameters, nn's can approximate complex functions arbitrarily well

they say if stacked non linear layers can approximate $H(x)$ they should also do $H(x) = x$

Core reformulation

↳ instead of learning $H(x)$ they learn $F(x) = H(x) - x$

network computes $F(x) + x$

Solvers might have difficulty approximating identity mapping with non linear layers

↳ meaning → it stacks like

layers → layer →

Deep Residual learning

$\mathcal{H}(x)$ → underlying mapping to be fit by a few stacked layers (not necessarily the entire net)

x → inputs

If one hypothesizes that multiple non linear layers can asymptotically approximate complicated functions

↳ then it is equivalent to hypothesize that they asymptotically approximate the residual functions $\mathcal{H}(x) - x$

(Assuming input & output have same dimensions)

So instead of expecting the stacked layers to approximate $H(x)$

we expect these layers to approximate $H(x) - x$

↳ then it is equivalent to hypothesize that they asymptotically approximate the residual functions $H(x) - x$ (Assuming input & output have same dimensions)

So instead of expecting the stacked layers to approximate $H(x)$ we expect these layers to approximate $H(x) - x$

The original function thus becomes $F(x) + x$

The degradation problem suggests that the solvers might have difficulties in approximating identity mappings, by multiple non linear layers

With residual learning reformulation, if identity mappings are optimal, the solvers may simply drive the weights of the multiple non linear layers toward zero to approach identity mappings

↳ in real cases there aren't optimal (unlikely) but the reformulation may help to precondition the problem

→ if optimal function is closer to an identity mapping than a zero mapping, it should be easier for the solver to find perturbations with reference to an identity mapping than to learn the function as new one

Residual functions in general have small responses, suggesting that identity mappings provide reasonable preconditioning

$y = F(x) + x$ creates a direct information path from input to output

fundamentally changing signal propagation through deep networks

Residual networks behave like

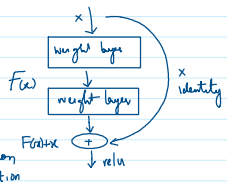
$x_L = x_0 +$ series of small updates

3.2 Identity mapping by shortcuts

Building block $y = F(x, \{W_i\}) + x$ (input vector)

y → output vector

$F(x, \{W_i\})$ → residual mapping to be learned



The operation $F + x$ is performed by a shortcut connection and element wise addition

the shortcut connections in above equation doesn't introduce extra parameter nor extra computational complexity

The dimensions of F & x must be equal

↳ if its not the case then (eg when changing input/output channels)

We can perform a linear projection of W_s by the shortcut connections to match the dimensions

$y = F(x, \{W_i\}) + W_s x$ → only used when mapping dimensions (a linear projection matrix)

3.3 Network Architecture

they compare · Plain network (CNN)

Res network (same CNN + shortcut connections)

- Core design rule

Rule 1 Same filters if feature map size is same

if spatial size stays the same

all layers have same no of filters eg 56×56 → conv 64 → conv 64

if feature map becomes smaller

they double the no of filters eg 56×56 → 28×28

56×56 conv 64

28×28 conv 128

When spatial size decreases, increasing channels keeps total computation roughly balanced

Downsampling → They reduce spatial resolution = 2 convolutions

Solvers might have difficulty approximating identity mapping with non linear layers

↳ meaning → A trace like

layer → layer →

has difficulty learning

$x_{l+1} = x_l$ even though identity seems simple (This is the optimization difficulty behind degradation)

If $H(x) = x$ is the best solution then $F(x) = 0$

Residual network can achieve this by making the weights of residual layers small

if $F(x) \approx 0$ the block output $y = x$ (easy for optimization)

Residual networks have small magnitude

↳ $F(x)$ usually has small magnitude

↳ meaning layers mostly learn small refinements, not large transformations

So the network behaves

↳ $x +$ small changes, small change +

28×28 conv128

Downsampling → They reduce spatial resolution = 2 convolutions

conv 3×3 stride 2 (this halves spatial size)

56×56 → 28×28

After conv blocks they use
global avg pooling
↓
fully connected layer
↓
softmax (1000 classes)

They take above network and add Residual connections

figure 4 left plot → plain networks — prove degradation problem
Right plot → with shortcuts — residual connections fix optimization issue

table 2 → proves the same

table 3/4 → performance keeps improving with the depth
depth becomes beneficial once optimization works

Bottleneck Architecture

For very deep networks they change the block structure

Instead of 3×3, 3×3 (kept) they use 1×1, 3×3, 1×1 } bottleneck block

purpose is to reduce computations and allow deeper networks

CIFAR-10 Experiments
they test very deep networks
ResNets can train 1000+ layers networks
plain networks count

ResNets → improve object detection performance

* plain deep nets get higher training error
* ResNets train much deeper models
* Once optimization works, deeper networks improve accuracy
* Layers mostly learn small refinements (Residual functions are small)
* Bottleneck design enables very deep models